Name: Sairamasubash Muppalaneni
NetID: smuppa3
Course: CS 538
Professor: Brighten Godfrey
Due Date: 11:59 pm CT, Wednesday May 3, 2023

CS 538 Assignment 2

Question 1:

(a)

Here is the FCT of the elephant flows and the FCT of the mouse flows:

- There are 4 elephant flows.
- Each elephant flow wants to download 6 GB files.
- In total it is 6 * 8 because there a total of 8 bits in a byte.
- The result is 48 GB.
- There are a total of 300 mouse flows.
- Each mouse flow arrives every 5 seconds
- The time is 60 seconds.
- There are 100 Mbps.
- We divide 100 Mpbs / 4 elephant flows.
- The result is 25 Mbps.
- Each elephant flow transfers 60 seconds * 25 Mbps of data.
- The result is 1,500 MB, which is equal to 1.5 GB.
- Now 4 elephant flows and 1 mouse flow joined together.
- The result is 5 total flows.
- We divide 5 total flows / 25 Mbps.
- The result is 0.2 seconds (which is the FCT of mouse flows).
- We divide 100 Mbps / 5 elephant and mouse flows.
- The result is 20 Mbps
- We multiply 20 Mbps * 0.2 seconds.
- The result is 4 MB.
- We subtract 5 seconds – 0.2 seconds.
- The result is 4.8 seconds.
- Now each elephant flow transfers 20 Mbps * 4.8 seconds of data.
- The result is 120 MB.
- We add 4 MB + 120 MB.
- The result is 124 MB.
- We multiply 300 mouse flows * 124 MB.
- The result is 37,200 MB, which is 37.2 GB.
- We subtract 48 total GB – 1.5 GB – 37.2 GB.
- The result is 9.3 GB, which is 9,300 MB.
- We divide 9,300 MB / 25 Mbps.

- The result is 372 seconds.
- We multiply 300 mouse flows * 5 seconds.
- The result is 1,500 seconds.
- So, we finally add 1,500 seconds + the original 60 seconds, and 372 seconds.
- The result is 1,932 seconds (which is the FCT of the elephant flows).

The FCT of the elephant flows is 1,932 seconds, and the FCT of the mouse flows is 0.2 seconds.

(b)

Here is the FCT of the elephant flows and the FCT of the mouse flows when mouse flows are always given priority over elephant flows:

- There are 4 elephant flows.
- Each elephant flow wants to download 6 GB files.
- In total it is 6 * 8 because there a total of 8 bits in a byte.
- The result is 48 GB.
- There are a total of 300 mouse flows.
- Each mouse flow arrives every 5 seconds
- The time is 60 seconds.
- There are 100 Mbps.
- We divide 100 Mpbs / 4 elephant flows.
- The result is 25 Mbps.
- Each elephant flow transfers 60 seconds * 25 Mbps of data.
- The result is 1,500 MB, which is equal to 1.5 GB.
- However, since the mouse flows have priority, we only use them first.
- So, we divide 1 mouse flow / 25 Mbps.
- The result is 0.04 seconds (which is the FCT of mouse flows).
- Now, since we are done with the mouse flows priority, we can go back to regular flows.
- We multiply 100 Mbps * 0.04 seconds.
- The result is 4 MB.
- We subtract 5 seconds – 0.04 seconds.
- The result is 4.96 seconds.
- Now each elephant flow transfers 100 Mbps * 4.96 seconds of data.
- The result is 120 MB.
- We add 4 MB + 120 MB.
- The result is 124 MB.
- We multiply 300 mouse flows * 124 MB.
- The result is 37,200 MB, which is 37.2 GB.
- We subtract 48 total GB – 1.5 GB – 37.2 GB.
- The result is 9.3 GB, which is 9,300 MB.
- We divide 9,300 MB / 25 Mbps.
- The result is 372 seconds.
- We multiply 300 mouse flows * 5 seconds.
- The result is 1,500 seconds.

- So, we finally add 1,500 seconds + the original 60 seconds, and 372 seconds.
- The result is 1,932 seconds (which is the FCT of the elephant flows).

The FCT of the elephant flows is 1,932, and the FCT of the mouse flows is 0.04 when mouse flows are always given priority over elephant flows.

(c)

With TCP in the real world, one reason that the mice might have higher FCT is because mouse flows are not that big. So, they might not have a lot of time to get to the highest possible rate. This leads to a higher FCT. With TCP in the real world, one reason that the mice might have lower FCT is because mouse flows have less packets. Due to this, the chance that a packet delivery that is out of order affecting them is much less. This allows the mouse flows to finish their transfers with a lower FCT (also, they experience lower delay when it comes buffering).

Question 2:

(a)

Even if all ASes (autonomous systems) use BGP with common business relationship policies, asymmetric routing can occur in the internet because there is a chance that there are different paths between two routers (load balancing). For example, let's say we have AS-A (customer of AS-B, and peer of AS-C), AS-B (customer of AS-C), and AS-C. If AS-A decides to send traffic to AS-C, even though there are 2 options, it uses the direct path to AS-C because it perfers the peer over the provider. If AS-C decides to send traffic to AS-A, even though there are 2 options, it uses the path through AS-B because it perfers the customer over the peer. This example takes into account the business relationship policies, and it shows asymmetric routing because traffic from AS-A goes to AS-C directly, while traffic from AS-C goes through AS-B to get to AS-A.

(b)

In a data center network with a leaf-spine topology, running shortest path routing with ECMP, when all links have the same length (cost), asymmetric routing can occur. I say this because of the hashing of the packet headers. There is not a 100% chance that the hashing of the forward direction is the same as the hashing of the backward direction.

Question 3:

(a)

Here is a sequence of events that lead to one stable state:

1. AS 1 anounces its IP prefix to AS 2 and AS 4.
2. Through the peer link, AS 4 passes on the announcement message to AS 3.
3. Through the provider to customer link, AS 3 passes on the announcement message to AS 2.
4. From the same IP prefix, AS 2 has now received 2 announcement messages. One from AS 1, and one from AS 3. Since, AS 1 is a backup that can only be used when no other path is available, AS 2 choses the route through AS 3.
5. To get to AS 1, AS 2 now sends its traffic through AS 3 (and uses that route).

(b)
Here is a different sequence of events that lead to a different stable state:

1. AS 1 anounces its IP prefix to AS 2 and AS 4.
2. Through the peer link, AS 4 passes on the announcement message to AS 3.
3. At this time, AS 2 has only received an announcement message (the IP prefix) from AS 1. Since AS 2 has no other path availabe, it sends its traffic through AS 1 (the backup).
4. Through the provider to customer link, AS 3 passes on the announcement message to AS 2.
5. At this time, AS 2 received an announcement message from AS 3, but it is already sending its traffic directly to AS 1 (so that route will continue to be used).

(c)
Here is the sequence of events that causes this story to happen:

1. To start of, the peer link fails between AS 3 and AS 4.
2. The announcement message that AS 3 sends to AS 2 is taken back.
3. The path that AS 2 uses through AS 3 to get to AS 1 is no more. So, AS 2 will now use the backup path, which is directly to AS 1.
4. Now, the peer link is restablished between AS 3 and AS 4.
5. The announcement message that AS 3 sends to AS 2 is sent once again.
6. At this time, AS 2 received an announcement message from AS 3 for the second time, but it is already sending its traffic directly to AS 1 (so that route will continue to be used).

Here we an example of the problem sometimes known as a BGP wedgie (start in (a), end in (b)).

Question 4:

(a)
Here is number of distinct end-to-end shortest paths in the physical topology between a source server A and a destination server B in a different pod:

- In the question, it says that there are 24-port switchs. So, k = 24.
- The paper mentioned in the question states that each pod has k/2 edge switches.
- The paper mentioned in the question states that each pod has k/2 aggregation switches.
- So, we do 12*12.
- The result is 144.

The number of distinct end-to-end shortest paths in the physical topology between a source server A and a destination server B in a different pod is 144.

(b)
Here is the probability that the TCP connection between A and B is interrupted (meaning that packets in either direction are dropped):

- As stated in the paper mentioned in the question, the total number of edge switches to aggregation switchs in the same pod is k * k/2 * k/2.
- We multiply 24 * 12 * 12.
- The result is 3,456.
- As stated in the paper mentioned in the question, the total number of aggregation switchs to core switches is k/2^2 * k/2.
- We multiply 12 * 12 * 12.
- The result is 1,728.
- Now we add 3,456 + 1,728.
- The result is 5,184.
- The number of distinct end-to-end shortest paths between a source server A and a destination server B is 144.
- We divide 144 / 5184.
- The result is 0.0278 (which is 2.78%).

The probability that the TCP connection between A and B is interrupted (meaning that packets in either direction are dropped) is 2.78%.

<u>Question 5:</u>

h = 567 km

timeM = 96 minutes

timeS = 5,760 seconds

R = 6371 km

<u>(a)</u>
Here is how fast the satellite's projection on the sea level is moving (in km/s):

- We multiply 2 * Pi * R, which is 2 * 3.14 * 6371 km.
- The result is 40,030.1736.
- Now we divide 40,030.1736 / timeS, which is 40,030.1736 / 5,760.
- The result is 6.9497.

The satellite's projection on the sea level is moving at a speed of 6.9497 km/s.

<u>(b)</u>
Here is the maximum length of contact between a satellite and a ground station:

- In total there are 360 degrees.
- The satellite has to be at least 15 degrees above the horizon.
- So, from the cosine rule, we get 25 degrees.
- We multiply timeM * 25, which is 96 * 25.

- The result is 2400.
- Now we divide 2400 / 360 degrees.
- The result is 6.6667.

The maximum length of contact between a satellite and a ground station is 6.6667 minutes.

<u>(c)</u>
Here is the speed of light:

3*10^8 m/s => 300,000,000 m/s

Here is the maximum signal propagation delay from satellite to ground station during a satellite-ground station contact:

- For the maximum signal propagation delay from satellite to ground station during a satellite-ground station contact, we need to add 567 km to 6371 km.
- The result is 6,938 km.
- Since the speed of light is m/s, we convert 6,938 km to 6,938,000 m.
- Then, we divide 6,938,000 m by 300,000,000 m/s.
- The result is 0.02313 s.

The maximum signal propagation delay from satellite to ground station during a satellite-ground station contact is 0.02313 s or 23.13 ms.

Here is the minimum signal propagation delay from satellite to ground station during a satellite-ground station contact:

- For the minimum signal propagation delay from satellite to ground station during a satellite-ground station contact, we just need to take 567 km.
- Since the speed of light is m/s, we convert 567 km to 567,000 m.
- Then, we divide 567,000 m by 300,000,000 m/s.
- The result is 0.00189 s.

The minimum signal propagation delay from satellite to ground station during a satellite-ground station contact is 0.00189 s or 1.89 ms.

<u>(d)</u>
Here are the number of satellites that we need to provide continuous coverage to the area beneath the orbit of the satellite:

- In total there are 360 degrees.
- From the cosine rule, we get 25 degrees.
- We divide 360 degrees / 25 degrees.
- The result is 14.4.
- However, we can have 40% of a satellite.

- So, we round up to 15 satellites.

The number of satellites that we need to provide continuous coverage to the area beneath the orbit of the satellite is 15 satellites.

<u>Question 6:</u>

For this question, I will choose the "Achieving High Utilization with Software-Driven Wan" (Hong, SIGCOMM 2013) paper, also known as SWAN.

<u>(a)</u>
The system's architecture in the SWAN paper and the system's architecture in the OpenFLow paper both use a centralized controller when it comes to managing the network resources and fowarding decisions. This part of the design is useful to the system because it provides a larger view of the state of the network, and it makes better decisions when it comes to traffic engineering/allocation of bandwith.

<u>(b)</u>
The system's architecture in the SWAN paper focuses on wide-area networks (WANs), and the system's architecture in the OpenFlow paper focuses on local-area networks (LANs). The purpose of this different design choice is to improve the utilization of wide-area network resources by taking into account the demands of traffic, the capacities of a link, and restrictions of a network.

<u>Question 7:</u>

<u>(a)</u>
Here is the probablility that the web server needs >= 200 milliseconds to be ready to reply to the client:

- The probability that all servers respond to the client in less than 200 milliseconds is 99.2%^120 database servers.
- The result is 0.3814 (which is 38.14%).
- The probability that the web server needs >= 200 milliseconds to be ready to reply to the client is 1 - 0.3814.
- The result is 0.6186 (which is 61.86%).

The probablility that the web server needs >= 200 milliseconds to be ready to reply to the client is 61.86%.

<u>(b)</u>
Here is the probablility that the web server needs to wait >= 200 milliseconds for its requests to complete:

- The probability that the 120 servers in phase 1 respond to the client in less than 200 milliseconds is 99.2%^120 database servers.

- The result is 0.3814 (which is 38.14%).
- The probability that the phase 1 web server needs >= 200 milliseconds to be ready to reply to the client is 1 - 0.3814.
- The result is 0.6186 (which is 61.86%).
- The probability that the 50 servers in phase 2 respond to the client in less than 200 milliseconds is 99.2%^50 database servers.
- The result is 0.6692 (which is 66.92%).
- The probability that the phase 2 web server needs >= 200 milliseconds to be ready to reply to the client is 1 - 0.6692.
- The result is 0.3308 (which is 33.08%).
- The probability that all 170 servers in phase 1 and phase 2 respond to the client in less than 200 milliseconds is 0.3814 times 0.6692.
- The result is 0.2552 (which is 25.52%).
- The probability that the web server needs to wait >= 200 milliseconds for its requests to complete is 1 - 0.2552.
- The result is 0.7448 (which is 74.48%).

The probablility that the web server needs to wait >= 200 milliseconds for its requests to complete is 74.48%.

References:

1. Course Slides
2. Course Readings
3. Scalable, Commodity DC Net Arch (Al Fares et al, SIGCOMM 2008)
4. OpenFlow (McKeown et al, 2008)
5. SWAN (Hong, SIGCOMM 2013)